SOFTWARE REQUIREMENTS SPECIFICATION

Student Portfolio Showcase

SRS • Project-specific requirements baseline

1. Introduction

Purpose: define functional, non-functional, interface and security requirements used for design, implementation, sprint planning, testing and evaluation. The word shall denote a mandatory requirement within the agreed scope.

1.1 Scope

Student Portfolio Showcase is a browser-based application through which an owner maintains profile information, skills, projects and achievements and publishes a public portfolio. Visitors can view published portfolios but cannot edit them. Proposed stack: Python 3.11, Flask, SQLite, HTML, CSS and JavaScript. GitHub Actions runs automated lint and test checks.

1.2 Intended audience

- Project developers and maintainers
- Testers and quality assurance contributors
- Course instructor and evaluators
- Users reviewing the intended system behaviour

1.3 Definitions

Term	Meaning
SRS	Software Requirements Specification
SAD	Software Architecture and Design
STP	Software Test Plan
CI/CD	Continuous Integration / Continuous Delivery or Deployment
Owner	Authenticated user who manages a portfolio
Visitor	Person viewing a published portfolio
RTM	Requirements Traceability Matrix

2. Overall description

The browser sends HTTP requests to Flask. The application validates input, applies authentication and ownership rules, and reads/writes portfolio records in SQLite. HTML templates and static assets provide the interface. pytest verifies behaviour; GitHub Actions runs lint and tests on pushes and pull requests.

2.1 Major product functions

- Register, log in and log out an owner (subject to final scope confirmation).
- Create and update owner profile information.
- Add, edit, list and remove skills, projects and achievements.
- Publish or unpublish a portfolio.
- Display published portfolio information to visitors.

- Validate input and return understandable errors.
- Persist user and portfolio data.
- Run automated linting and tests through GitHub Actions.

2.2 User classes

Class	Needs
Portfolio owner	Manage profile, skills, projects, achievements and publication status
Visitor	View published information without editing
Developer/maintainer	Run application/tests, review code and maintain CI
Evaluator	Inspect requirements, repository and working demonstration

2.3 Constraints and environment

- Python 3.11; Flask; SQLite; HTML/CSS/JavaScript; modern browser.
- Use one shared project repository as instructed by the course.
- Use branches and pull requests for team changes; review before merging.
- Never commit secrets, tokens or real credentials.
- Hosting provider and public deployment requirement are pending confirmation.

3. External interface requirements

3.1 User interface

- Provide navigation for profile, skills, projects, achievements and public portfolio.
- Label form fields and show validation feedback clearly.
- Keep core flows usable on common desktop and mobile viewports.
- Do not expose editing controls to public visitors.

3.2 Software and communication interfaces

- Flask routes provide browser-facing and API behaviour; SQLite stores initial data.
- pytest executes automated tests; Ruff performs configured lint checks.
- GitHub Actions runs lint and tests on pushes to main and pull requests targeting main.
- Local browser-to-server communication may use HTTP; any public deployment shall use HTTPS.
- API endpoints use JSON where defined; browser routes may return HTML.

4. Functional requirements

ID	Feature	Requirement	Priority	Test ID
SPS-FR-001	Registration	Allow a new user to create an account using required fields.	High	TC-AUTH-01
SPS-FR-002	Login	Authenticate a registered user using valid credentials.	High	TC-AUTH-02
SPS-FR-003	Invalid login	Reject invalid credentials without identifying which credential was wrong.	High	TC-AUTH-03
SPS-FR-004	Logout	Invalidate the active session when an owner logs out.	High	TC-AUTH-04
SPS-FR-005	Create profile	Allow authenticated owner to create display name, headline, biography and links.	High	TC-PROF-01
SPS-FR-006	Edit profile	Allow owner to update their own profile and persist changes.	High	TC-PROF-02
SPS-FR-007	View profile	Display profile fields selected for publication.	High	TC-PROF-03

SPS-FR-008	Add skill	Allow owner to add a skill with name and optional category/proficiency.	Medium	TC-SKILL-01
SPS-FR-009	Edit/remove skill	Allow owner to edit or remove their own skills.	Medium	TC-SKILL-02
SPS-FR-010	Add project	Allow owner to add title, description, technology tags and optional URLs.	High	TC-PROJ-01
SPS-FR-011	Edit project	Allow owner to update their own project details.	High	TC-PROJ-02
SPS-FR-012	Remove project	Allow owner to remove a project after explicit action.	Medium	TC-PROJ-03
SPS-FR-013	Achievements	Allow owner to add, edit and remove achievements/certifications.	Medium	TC-ACH-01
SPS-FR-014	Publish	Allow owner to publish or unpublish their portfolio.	High	TC-PUB-01
SPS-FR-015	Public view	Provide shareable public view for published portfolios without edit controls.	High	TC-PUB-02
SPS-FR-016	Private restriction	Prevent visitors from viewing unpublished/private portfolios.	High	TC-PUB-03
SPS-FR-017	Validation	Validate required fields, lengths and URL formats before storage.	High	TC-VAL-01
SPS-FR-018	Errors	Return appropriate status codes or user-facing errors for invalid/missing/unexpected conditions.	High	TC-ERR-01
SPS-FR-019	Persistence	Persist user and portfolio records across application restarts.	High	TC-DB-01
SPS-FR-020	Health	Expose a health endpoint indicating application responsiveness.	Medium	TC-OPS-01
SPS-FR-021	CI checks	Run configured lint and tests on pushes to main and PRs targeting main.	High	TC-CI-01
SPS-FR-022	CI failure	Report failure when lint or automated tests fail.	High	TC-CI-02

5. Non-functional requirements

ID	Category	Requirement	Priority
SPS-NFR-001	Performance	Common local demo requests should complete within 2 seconds at p95, excluding external network delays.	Medium
SPS-NFR-002	Usability	First-time user can locate profile, projects and skills using labelled navigation.	Medium
SPS-NFR-003	Reliability	Invalid input and missing records do not terminate the server.	High
SPS-NFR-004	Maintainability	Organise application logic into understandable modules and keep tests separate.	High
SPS-NFR-005	Compatibility	Core flows usable in current Chrome and Firefox versions.	Medium
SPS-NFR-006	Accessibility	Use meaningful labels, keyboard-operable controls and sufficient contrast.	Medium

SPS-NFR-007	Portability	Developer can install, test and run using documented steps.	High
SPS-NFR-008	CI feedback	Normal lint-and-test workflow should complete within five minutes.	Medium

6. Security requirements

ID	Area	Requirement
SPS-SEC-001	Password handling	If password authentication is implemented, store only salted hashes; never store/log plaintext passwords.
SPS-SEC-002	Access control	Every private create/update/delete verifies authenticated user owns the resource.
SPS-SEC-003	Session protection	Use appropriate session cookie settings; production cookies Secure and HttpOnly, SameSite configured.
SPS-SEC-004	Input/query safety	Validate and constrain input; use parameterised database operations or ORM.
SPS-SEC-005	Secrets	Do not commit secrets/tokens; use environment variables or hosting secret store.
SPS-SEC-006	Error/log hygiene	Do not disclose passwords, session secrets or unnecessary stack traces.
SPS-SEC-007	Transport	Any public deployment shall use HTTPS.

7. Acceptance criteria

ID	Area	Criterion	Verification
SPS-AT-001	Startup	Documented command launches Flask without errors.	README walkthrough
SPS-AT-002	Health	GET /health returns HTTP 200 and healthy payload.	pytest
SPS-AT-003	Profile	Owner creates/updates profile and changes persist.	Functional test
SPS-AT-004	Portfolio entries	Owner manages supported entries.	CRUD tests
SPS-AT-005	Publication	Published public; unpublished private.	Access-control tests
SPS-AT-006	Validation	Invalid inputs rejected without corrupting data.	Negative tests
SPS-AT-007	CI	PR triggers checks; failing check reported failed.	Test PR
SPS-AT-008	Documentation	New developer can install, test and run.	Clean setup walkthrough

8. Use-case overview

Use case	Actor	Precondition	Outcome
Register/login	Owner	Not authenticated	Session established or error shown
Manage profile	Owner	Authenticated	Profile stored and previewable
Manage entries	Owner	Authenticated	Skills/projects/achievements changed
Publish portfolio	Owner	Portfolio exists	Public/private state updated
View portfolio	Visitor	Portfolio published	Visitor sees public content only
Validate code	Developer	Push/PR occurs	CI reports lint/test status

9. Traceability

Requirement group	Design mapping	Verification	Sprint
SPS-FR-001–022	Flask routes/services and data models (proposed)	TC IDs in Section 4	Assign in Jira after team planning
SPS-NFR-001–008	Architecture and implementation controls	STP quality checks	Assign in Jira
SPS-SEC-001–007	Application/session/database configuration	STP security checks	Assign in Jira